

- LLM 업무 자동화 4-Layer 방법론
 - 핵심 원칙
 - Layer 1: 컨텍스트 DB
 - Layer 2: 도구(스크립트) 라이브러리
 - Layer 3: 역할 프롬프트 (시스템 프롬프트)
 - Layer 4: 스케줄러 (자동 실행 엔진)
 - 트리거 유형
 - 설계 원칙
 - 아키텍처 흐름
 - 프로젝트에 세팅하는 순서
 - 1단계: 서비스의 Write 작업 전수조사
 - 2단계: DB 스키마 설계
 - 3단계: 핵심 도구 20개 구현
 - 4단계: 역할 프롬프트 초안 작성
 - 5단계: 실사용하면서 도구 추가 + 프롬프트 수정 반복
 - 6단계: 가장 자주 손으로 시키는 작업 3개를 스케줄러로 전환
 - 7단계: 안정화 후 스케줄러 점진적 확대 + 실행 로그 모니터링 체계 구축
 - 에이전트의 본질: Write

LLM 업무 자동화 4-Layer 방법론

정리일: 2026-04-30

어떤 프로젝트든, 어떤 LLM이든 — 이 네 겹만 깔면 사람 없이도 일관된 결과가 나온다.

핵심 원칙

LLM이 하는 건 "판단"뿐이다. 기억은 DB가, 실행은 도구가 한다. 그래서 LLM을 교체해도, 세션이 바뀌어도, 일관되고 균질한 결과가 나온다.

Layer 1: 컨텍스트 DB

LLM의 "기억"을 LLM 안에 두지 않는다. 외부 DB에 구조화해서 넣는다.

왜? LLM은 대화가 바뀌면 맥락을 잃는다. 세션이 끊기면 처음부터 다시. 근데 DB에 있으면 어떤 세션에서든 같은 맥락을 꺼내쓸 수 있다. LLM을 바꿔도 맥락은 그대로.

구체적으로 뭘 넣나:

- 고객/프로젝트 정보 (누가, 뭘, 언제까지)
- 의사결정 히스토리 (왜 이렇게 했는지)
- 도메인 지식 (업계 용어, 규칙, 제약조건)
- 진행 상태 (지금 어디까지 왔는지)

핵심은 "사람이 요약해서 넣는 게 아니라, 구조화된 테이블로 넣는 것". 프리텍스트 메모가 아니라 쿼리 가능한 데이터.

Layer 2: 도구(스크립트) 라이브러리

반복되는 작업을 사전에 도구로 만들어놓는다. LLM은 "어떻게 할지" 고민하지 않고, "어떤 도구를 쓸지"만 판단한다.

왜? LLM이 매번 처음부터 코드를 짜거나 절차를 생각하면 — 할 때마다 결과가 다르다. 근데 검증된 스크립트를 실행만 하면 항상 같은 결과가 나온다.

도구 설계 원칙:

- 도구 하나 = 행위 하나 (일정 등록, 파일 파싱, 데이터 저장 등)
- 입력/출력이 명확하게 정의됨
- LLM이 파라미터만 채우면 실행 가능
- 처음부터 전부 만들 필요 없음. 핵심 20개로 시작하고, 쓰다 보면 빠진 게 보인다

MCP(Model Context Protocol) 같은 표준을 쓰면 도구를 LLM-agnostic하게 만들 수 있다. Claude든 GPT든 Gemini든 같은 도구를 호출.

Layer 3: 역할 프롬프트 (시스템 프롬프트)

"컨텍스트를 어떻게 읽고, 도구를 언제 쓸지"의 판단 기준을 정의한다.

포함할 것:

- 역할과 목표 ("너는 ~이고, ~를 한다")

- 판단 기준 ("이런 상황에서는 이렇게 행동한다")
- 톤/형식 ("이런 식으로 답한다")
- 제약 조건 ("이건 하지 않는다")

핵심은 **버전 관리**. 사용자 피드백이 오면 프롬프트를 수정하고, 수정 이유를 기록한다. 프롬프트가 진화하면 LLM의 행동도 진화한다.

Layer 4: 스케줄러 (자동 실행 엔진)

Layer 1~3이 "시키면 잘 하는 구조"라면, Layer 4는 "시키지 않아도 알아서 하는 구조"다.

왜 필요한가? 사람이 매번 "브리핑해줘", "데이터 업데이트해줘"를 말해야 한다면 — 그건 도구를 잘 쓰는 거지, 자동화가 아니다. 진짜 자동화는 정해진 시점에 정해진 작업이 사람 개입 없이 실행되는 것.

트리거 유형

- 시간 기반: "매일 9시", "매주 월요일" → cron/launchd
- 이벤트 기반: "파일이 들어오면", "API 응답이 오면" → webhook/listener
- 조건 기반: "주가가 X% 넘으면", "마감일 3일 전이면" → polling + 조건 체크

설계 원칙

1. **작업 단위 = Layer 2의 도구 조합** — 스케줄러가 직접 복잡한 로직을 돌리는 게 아니라, 기존에 만들어놓은 도구들을 순서대로 호출하는 것
 2. **LLM 호출 포함 가능** — 판단이 필요한 반복 작업은 스케줄러가 LLM을 호출해서 Layer 3(역할 프롬프트) + Layer 1(DB 컨텍스트)을 태워서 실행
 3. **실행 로그 필수** — 매 실행마다 시작 시각, 성공/실패, 처리 건수, 에러 메시지를 기록. 실패 시 알림(슬랙, 텔레그램 등)을 쏘는 것까지가 세트
 4. **점진적 확대** — 가장 자주 손으로 시키는 작업 3개부터 자동화하고, 안정되면 하나씩 추가
-

아키텍처 흐름

[Layer 4] 스케줄러 - 언제 실행할지
↓
[Layer 3] 역할 프롬프트 - 어떻게 판단할지
↓
[Layer 1] 컨텍스트 DB - 무엇을 참조할지
↓
[Layer 2] 도구 라이브러리 - 무엇을 실행할지
↓
결과 저장 (→ Layer 1 DB) + 알림 (→ 사용자)

사람이 하는 건 "어떤 작업을 얼마나 자주 돌릴지" 설정하는 것뿐. 나머지는 시스템이 순환한다.

프로젝트에 세팅하는 순서

1단계: 서비스의 Write 작업 전수조사

- 뭘 생성/수정/삭제하는지 전부 매핑
- 누락된 작업이 하나라도 있으면 에이전트가 그 지점에서 "못 합니다"를 뱉고, 사용자 신뢰가 깨진다
- 100% 커버가 목표

2단계: DB 스키마 설계

- 컨텍스트를 어떤 구조로 저장할지 결정
- 프리텍스트가 아니라 쿼리 가능한 테이블 구조
- 의사결정 히스토리, 도메인 지식, 진행 상태 포함

3단계: 핵심 도구 20개 구현

- 가장 자주 쓰는 작업부터
- 도구 하나 = 행위 하나 원칙
- MCP 표준 사용 시 LLM-agnostic

4단계: 역할 프롬프트 초안 작성

- 역할, 판단 기준, 톤, 제약 조건 정의
- 버전 관리 체계 구축

5단계: 실사용하면서 도구 추가 + 프롬프트 수정 반복

- 빠진 도구를 추가하고, 프롬프트를 다듬는 반복 사이클
- 피드백 → 프롬프트 수정 → 행동 개선의 자기 진화 루프

6단계: 가장 자주 손으로 시키는 작업 3개를 스케줄러로 전환

- 시간 기반(cron/launchd) 또는 이벤트 기반(webhook) 선택
- 기존 도구(Layer 2)를 조합하여 자동 실행

7단계: 안정화 후 스케줄러 점진적 확대 + 실행 로그 모니터링 체계 구축

- 실행 로그 대시보드, 실패 알림 채널 구축
- 신규 자동화 작업을 지속적으로 추가

에이전트의 본질: Write

Read만 하는 AI는 검색엔진이다. "일정이 뭐 있어?"에 답하고, "이 문서 요약해줘"를 처리하는 건 에이전트가 아니다.

Write가 되는 순간 에이전트가 된다. 일정을 등록하고, 문서를 작성하고, 주문을 넣고, 상태를 변경하는 것. 사용자가 "해줘"라고 했을 때 실제로 해지는 것.

에이전트 = MCP(서비스 완전 커버) × 역할(명확한 목표) × 컨텍스트(실시간 상황) × 피드백(자기 진화)